

CS319: Scientific Computing (with C++)  
**PROJECTS (Version 31.03.2026a)**

Niall Madden

Week 8, Semester 2, 2025/2026

<https://www.niallmadden.ie/2526-CS319/2526-CS319-Projects.pdf>

- 1** Overview
  - Teams of 1 or 2
  - Time-line and grading
- 2** Project topics
  - Ideas
- 3** Project proposal
- 4** The code
  - Submitting code
- How code will be assessed
- 5** The report
- 6** Academic Conduct; Ethical use of Gen-AI
  - Examples
- 7** Presentations
  - Outline
  - Schedule (final)

# 1. Overview

Your project will consist of

1. **Topic:** This is an idea you select from some of those listed below, and is agreed in an **in-person discussion** with Niall in labs or lectures. You can also propose your own topic, but it must have a clear link to scientific computing.
2. **Proposal:** An outline for what you plan to achieve. This will be presented as 250 word project proposal. (Word count is indicative: I won't check).
3. **C++ Code:** Your own implementation of some algorithm, coded in C++ (or C++ and Python), and with your own choice of test problem(s) so you can verify properties of the algorithm, such as its complexity, or convergence, etc.
4. **Report:** ideally written as Jupyter notebook, which explains the problem, describes the algorithm, and draws conclusions about its properties.
5. **Presentation:** lightning talk on your work, including a demo of your code. Alternative *reasonable accommodations* (with supporting LENS report) can be discussed, such as a video presentation.

- ▶ Most projects are individual, but you are welcome to partner with one other person (assuming they agree!).
- ▶ For paired projects, you have to find a partner. Then both of you have to let me know.
- ▶ Both members will get the same grade for all parts jointly done.

Your projects will contribute 40% to your over-all grade for CS319. The break-down of marks is as follows.

Note: Unless otherwise stated, deadlines are at 17:00 on the day mentioned.

| What                     | When                             | Marks |
|--------------------------|----------------------------------|-------|
| Project topic negotiated | Tuesday, 10 March (W09)          | 3     |
| Project proposal         | 13:00, <b>Wed 18 March (W10)</b> | 5     |
| Presentations            | 1st and 2nd April, (W12)         | 7     |
| C++ Code (progress)      | Friday, 27 March                 | 2     |
| C++ Code                 | Friday, 10 April                 | 14    |
| Project report           | Friday, 10 April                 | 9     |

- ▶ Proposals and final reports will be checked with TurnItIn;
- ▶ All code will be checked manually and by computational tools to verify good academic conduct.
- ▶ You can expect contact from me seeking clarification on any issues arising with the code (e.g., it does not compile; looks like it was taken from elsewhere; suspected use of Gen-AI).
- ▶ Any issues of academic conduct will be logged on the Academic Misconduct Register.

## 2. Project topics

Next, I'll outline a selection of topics you might consider working on. You can propose your own ideas, but they must relate to scientific computing.

**You are required to discuss your ideas with me in the lab** tomorrow (5 March) or Friday (6 March)!

Failing that, can discuss by email (though not for full marks, and you will still have to discuss the proposal in person).

**Deadline:** submit your agreed project topic by **17:00, Tuesday, 10 March**.

### Some ideas for projects

*The topics listed here have the purpose of stimulating discussion. More will be added to this as I think of them. No two people/teams will be allocated the same topic.*

*Topics can involve standard algorithms in a new setting (e.g., applications of methods studied in class), or new algorithms (maybe to standard examples). Don't assume a topics below is available. Also: the “...” ones are place-holders for ones suggested in class.*

1. **OOP**: Your own implementation of mixed precision floats, with applications and testing. (Very suitable for group)
2. **Programming**: ~~Parallel computing with OpenMP for quadrature [TD]~~
3. **Programming**: ~~Felsenstein's Pruning Algorithm with OpenMP [MR]~~

4. **OOP**: Your own implementation of arbitrary precision integers, with applications, e.g., in cryptography or primality testing.
5. **OOP**: Piecewise polynomials: implementation and application.
6. **Optimization**: ~~Gradient descent in (at least) two dimensions. [AB]~~
7. **Optimization**: ~~Particle Swarm [RD]~~
8. **Optimization**: ~~Monte Carlo Methods, with applications to graphics. [AJ]~~
9. **Optimization**: ~~Monte Carlo Methods, with applications to travelling sales man [JA]~~
10. **Data Science**: ~~Nonnegative matrix factorization, with applications [HG]~~
11. **Data Science**: ~~Clustering via  $k$ -means, etc [SL]~~



- 12. **Data Analysis:** ~~SVD (=PCA) with applications in data science and/or image processing. [SM]~~
- 13. **Functional Data Analysis:** Interpolation of smooth data with *adaptive* error control.
- 14. **Functional Data Analysis:** ~~Functional approximation of noisy data with smoothing [CH]~~
- 15. **Interpolation:** Interpolation and sampling of functions in 2D.
- 16. **Interpolation:** ~~Convexity Preserving Methods, such as rational splines; NURBS (probably in 1D) [GH]~~
- 17. **Quadrature:** ~~Numerical integration with adaptive error control [MK]~~
- 18. **Quadrature:** Very high-dimensional numerical integration
- 19. **Graphs/Networks:** Spectral clustering.

- 20. **Networks**: Just about any algorithm from CS4423.
- 21. **Numerical Analysis**: Any topic from MA385 or MA378.
- 22. **Linear algebra**: ~~Computing the Choleskey factorisation of a matrix, and explain its applications in solving linear systems and/or determining positive definiteness [MG]~~
- 23. **Linear algebra**: Conjugate Gradients (and/or GMRES), and show how and when it works (and fail).
- 24. **Linear algebra**: Schur compliments for solving linear systems.
- 25. **Linear algebra**: ~~QR Factorization and applications [SMcD]~~
- 26. **Differential equations**: ~~Solution of boundary value problems by finite difference methods [MK]~~
- 27. **Other**: ~~Logic gates [EÓG]~~

- 28. **Other:** Needleman-Wunsch algorithm for global sequence alignment [TMcD]
- 29. **Suggestions?** Make clear application area, and, especially, the algorithms to be used.

### 3. Project proposal

- ▶ The project proposal outlines what you plan to achieve.
- ▶ It should begin with a (tentative) title, your name, ID and email address, and the name of your collaborator (if any).
- ▶ The proposal should describe what you plan to do, explain why it is interesting, and describe how you plan to test it.
- ▶ It should list any major classes you plan to design and implement. It is very useful to give the function prototypes for the major functions you plan to write.
- ▶ It should include a set of milestones, and a time-line for each. (It more important that you make such as plan, than actually following the time-line!)
- ▶ 250 words should suffice. (Word count is indicative: I won't check).
- ▶ This will be uploaded to Canvas as a PDF.
- ▶ **Deadline: 13:00, Wednesday 18 March.**
- ▶ If your proposal is OK (and on time), you get full marks for that part. If not, you can resubmit until you do. (That is: we want to get this right).

## 4. The code

The code for your project is a chance for you to show your skills in C++ programming, and in scientific computing.

Examples of things you might try to demonstrate:

- ▶ skill in writing modular code. At the very least there should be one function, but I expect there will be multiple functions.
- ▶ is your code *efficient*? Have you identified any bottlenecks?
- ▶ examples of writing your own class will be very welcome.
- ▶ can you use an external data source or produce one (and show skill in data input/output).
- ▶ **Reuse unattributed or AI-generated code will leave to Academic Integrity procedures being invoked.**

1. Code should be submitted on Canvas (see [Project](#) module). If you prefer to share it via a git repository, that would be very welcome. Make sure the repo is private and shared with me. In addition, add a link to it as text in the code section on Canvas.
2. If your code features multiple files (which is very likely), you may submit them as a [zip](#) file, or similar archive.
3. Any algorithms should be implemented in C++ ([.cpp](#)). You can use Python ([.py](#)) code or Jupyter notebooks ([.ipynb](#)) for visualisation or post-processing.
4. Every **code** file you submit (other than 3rd party libraries) should, without exception, have comments at the start with
  - your name, ID and email address
  - comments that briefly summarising what the code does and how to use it.

You are not required to add you name, etc, to data files that are part of the project.

5. There is no lower or upper limit in the number of lines your code must have. My experience is that good projects often feature about 200 lines of code for algorithms, functions, classes, etc, and about 100 lines of code for demonstrating that it works.
6. Code that you claim is written by you should be written by you. Don't use anyone else's code without attribution. *If plagiarism is identified in a project, Academic Integrity procedures will be invoked.*
7. Don't submit AI-generated code. Just don't. The project is worth 40% of the grade for the module, so penalties permitted include scoring zero on the module, with no opportunity to re-sit.

Broadly, your code will be assessed for the extent for which it shows your programming in skills in C++ (as well as Python, if appropriate).

In doing that, I will pay particular attention to the following:

- ▶ Is there competent use of basic programming structures, such as `for` loops and `if` statements?
- ▶ Is there appropriate use of data-types, with conversion between them as needed?
- ▶ Have you shown you can work with external data sources, e.g., reading data from text files, or importing from images or spread-sheets, etc.
- ▶ **Important:** demonstrated skills in scientific computing, including the implementation of algorithms, and correct use of tools.
- ▶ **Important:** Your project should have at least one function, ideally several/many.
- ▶ You can use external libraries, providing your code shows that you have learned lots about C++.



## 5. The report

Your report will be submitted as a PDF document, ideally (but not necessarily) generated from a Jupyter notebook.

It should start with:

- ▶ project title
- ▶ your name, ID number, email address
- ▶ names of anyone you collaborated with.

Suggested organisation:

1. **Introduction:** A summary of what you have done. This should be 300-500 words, and written in a non-technical style: anyone should be able to understand it. The emphasis should be on the problem solved, and why it is interesting, and not on the code.

## 5. The report

2. **Code:** A technical discussion of the code. Outline the major algorithms you've implemented, functions you've written, or classes you've developed. This can be quite “high-level”: you don't have to explain what individual lines of code do, but rather focus on the big picture: *what was the main challenge to be addressed?*  
However, **if you code makes use of any aspect of the C++ language that was not covered in class, you should explain that.** If you may present this part as a video with a walk-through of the code.
3. **Case use:** An example of typical input and output. Having read this, the use should know enough to test your code for their own examples.
4. **Discussion:** A discussion on what you have learned/discovered. What have you discovered about this algorithm and/or this method? **What steps have you taken to verify its properties?** For example, if relevant, have you established the rate of convergence, or estimated the run-time for inputs of given sizes?

## 5. The report

5. **Highlights:** What are the highlights of your project? What took the most effort? What are you most proud of?
6. **Conclusion:** Details any limitations of your code that you have noted, and a comparison with your original project proposal. If you had more time to work on your project, what would you do next? List any references used.

Submit a PDF version of the report (which will be checked with TurnItIn). You can also upload a Jupyter notebook, if you think it would be useful.

**Important:** you are also welcome to upload a short video explaining how your code works. This might be particularly useful if your code contains any constructs that were not covered in class.

## 6. Academic Conduct; Ethical use of Gen-AI

You are prohibited from submitting any code obtained by Gen-AI (*no vibe coding!*). However, this does not prohibit you from using Gen-AI as part of your work flow, e.g., as a search engine. If in doubt, ask.

- ▶ **Acceptable:** asking Gen-AI general questions on syntax and, especially, 3rd party libraries.
- ▶ **Unacceptable:** Submitting any AI-generated code.
- ▶ **Acceptable:** asking Gen-AI to proof-read your report, to identify typos.
- ▶ **Unacceptable:** Submitting any AI-generated text.
- ▶ **Acceptable:** asking Gen-AI for help producing graphics for your text (such as an illustrative cartoon, or helping with Python code to analyse your results).
- ▶ **Unacceptable:** uploading *any* resources I provide to a generative AI system.

Any use of AI should be fully acknowledged, including the model and prompts used.

## 6. Academic Conduct; Ethical use of Gen-AI Examples

- ▶ In preparing some lecture notes, I use co-pilot to help create diagrams in a language called “tikz”, which I am not very proficient in. I give it an example, and ask how to improve it.
- ▶ I made the logo for these projects using co-pilot with the prompt: `Make a logo for 'CS319 Project' with the title in bold tech-style font. Below it, add smaller {C++} in a code-like style.`



Generated with Co-pilot

- ▶ Any suspected instance of academic misconduct will be logged using the handy new [Academic Misconduct Register!](#)

- ▶ Presentations will take place in Week 12.
- ▶ Each presentation should feature at most 3 slides. The topics might be
  - My project topic
  - What I've learned so far
  - What I plan to do next
- ▶ **You should show some code**, even if not running. For example, you could include a snippet of code with a class or function prototypes. If you want, you can demonstrate the code running (providing you are confident you can connect to the projector).
- ▶ Presentations should take at most 4 minutes. Each will be followed by a question, with another minute to answer.
- ▶ Presentations should be uploaded to Canvas no later than 13.00 on Wednesday, 25 March. Ideally, the presentation should be in PDF. If using PowerPoint (or equivalent) keep it simple.

## SCHEDULE

Times are approximate. Talks may start earlier than advertised. You should attend the Wednesday session, even if not presenting, and, if possible, the Thursday one too.

| Wednesday, 1 April |              |       |
|--------------------|--------------|-------|
|                    | Who          | When  |
| 0                  | Cassie H     | 16.00 |
| 1                  | Michael K    | 16:06 |
| 2                  | Hiruki G     | 16:12 |
| 3                  | John A       | 16:18 |
| 4                  | Theo Ó D     | 16:24 |
| 5                  | Aleksandra B | 16:30 |
| 6                  | Sean L       | 16:42 |

| Thursday, 2 April |           |       |
|-------------------|-----------|-------|
|                   | Who       | When  |
| 0                 | Mohit R   | 09.00 |
| 1                 | Máire G   | 09.06 |
| 2                 | Megan L   | 09.12 |
| 3                 | Adam J    | 09.18 |
| 4                 | Gavin H   | 09.24 |
| 5                 | Sean McD  | 09.30 |
| 6                 | Shay M    | 09.36 |
| 7                 | Eunan Ó G | 09.42 |